

Internship Assessment Report

On

“ARTIFICIAL INTELLIGENCE”

from

“CodTech IT Solutions Pvt. Ltd..”

in partial fulfilment of the requirements

for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Shani Gangwar | 2304500100097

Submitted to

Mr. Harshit Yadav

Assistant Professor



Department of Computer Science and Engineering

Shri Ram Murti Smarak College of Engineering Technology & Research, Bareilly

Dr. A. P. J. Abdul Kalam Technical University, Lucknow

July, 2026

CERTIFICATE

INTERNSHIP



CODTECH IT SOLUTIONS PRIVATE LIMITED

8-7-7/2, Plot NO. 51, Opp: Naveena School, Hasthinapuram Central, Hyderabad,
500 079. Telangana

CERTIFICATE

OF INTERNSHIP EXPERIENCE



To whom so ever it may concern

Intern Name: Shani Gangwar

Intern ID: CITS2886

Program Type: Internship

Duration: 6 Weeks

Domain: Artificial Intelligence

Organization: Codtech IT Solutions Private Limited

Internship Period: 06 June 2026 - 18 July 2026

Status: Successfully Completed

Remarks: Demonstrated strong dedication, technical skills, and a positive learning attitude throughout the internship.

We appreciate his/her active participation, consistent efforts, and impressive contribution to the overall success of the internship. We wish him/her all the best in future endeavors.

We are confident that his/her dedication and skills will lead to great achievements ahead.

Best Wishes,
NEELA SANTHOSH KUMAR
Human Resources & Academic Head
Hr@codtechitsolutions.com



#startupindia

TABLE OF CONTENTS

	Page No.
ACKNOWLEDGEMENT.....	iii
LIST OF TABLES.....	iv
LIST OF FIGURES.....	v
LIST OF ABBREVIATIONS	vi
CHAPTER 1. INTRODUCTION TO COMPANY	1
CHAPTER 2. TECHNOLOGY LEARNED	2
CHAPTER 3. INTRODUCTION TO PROJECT	3
3.1 Project Background	3
3.2 Project Methodology	5
3.3 Objective	7
3.4 Problem Statement	8
3.5 Motivation	9
3.6 Literature Review	10
3.7 Requirement Analysis	11
CHAPTER 4. RESULT & DISCUSSION	22
4.1 Result	22
4.2 Output	23
4.3 Discussion	24
4.4 Application	25
CHAPTER 5. FUTURE ENHANCEMENT	26
5.1 Conclusion	27
CHAPTER 6. REFERENCES	28

ACKNOWLEDGEMENT

It gives me immense pleasure to present this internship report, completed during the B.Tech. summer internship after the VI Semester.

I express my sincere gratitude to Mr. Santhosh Kumar, HR and Training Head, CodTech IT Solutions Pvt. Ltd., Hyderabad, for providing me with the opportunity to undertake this internship. His continuous guidance, encouragement, valuable suggestions, and constant support throughout the internship greatly contributed to the successful completion of my project. His dedication, knowledge, and motivation inspired me to learn and apply practical concepts in the field of Artificial Intelligence.

I would also like to express my heartfelt gratitude to the faculty members of the Department of Computer Science and Engineering for their valuable guidance, encouragement, and support throughout the internship period.

Finally, I would like to thank my friends and colleagues for their cooperation, encouragement, and assistance during the completion of this project. Their support played an important role in the successful completion of my internship.

I sincerely thank everyone who directly or indirectly contributed to making this internship a valuable learning experience.

Signature - _____

Roll No – 2304500100097

Name – Shani Gangwar

Date - ____/____/____

LIST OF TABLES

Table 1. Project Overview.....	4
Table 2. Project Development Methodology	6
Table 3. Project Objectives.....	7
Table 4. Functional Requirements.....	12
Table 5. Non-Functional Requirements.....	13
Table 6 Overall System Results.....	24
Table 7 Query Match Score.....	24

LIST OF FIGURES

Figure 1. Overall Working.....	4
Figure 2. Methodology Diagram.....	6
Figure 3. Use Case Diagram	15
Figure 4. Flow Chart of System.....	16
Figure 5. API Testing (Input).....	22
Figure 6. API Testing(Output)	22
Figure 7. User UI/UX.....	25

LIST OF ABBREVIATION

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
ML	Machine Learning
FAQ	Frequently Asked Questions
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
UI	User Interface
UX	User Experience
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
DBMS	Database Management System
SQL	Structured Query Language
CRUD	Create, Read, Update and Delete
PDF	Portable Document Format

CHAPTER 1

Introduction to Company

CodTech IT Solutions Private Limited is an emerging technology company dedicated to delivering innovative software solutions, professional IT training, and industry-oriented internship programs. The company focuses on empowering students and professionals by bridging the gap between academic learning and practical industry requirements. Through its commitment to excellence, SkillInfyTech provides hands-on experience in modern technologies such as **Artificial Intelligence (AI), Machine Learning (ML), Data Science, Full-Stack Web Development, Cloud Computing, Cybersecurity, and Mobile Application Development**. By combining theoretical knowledge with real-world project implementation, the company prepares aspiring professionals to meet the evolving demands of the IT industry.

CodTech offers a comprehensive range of services, including custom software development, web and mobile application development, AI-powered solutions, cloud-based applications, technical consulting, and professional certification programs. The company also conducts internship and training programs where students work on live industry projects under the guidance of experienced mentors, enabling them to gain practical exposure to the complete Software Development Life Cycle (SDLC), from requirement analysis and system design to development, testing, deployment, and maintenance.

The organization's mission extends beyond technical education by fostering innovation, problem-solving abilities, teamwork, and professional ethics among learners. It emphasizes the development of both technical expertise and essential soft skills, including communication, analytical thinking, and project management. With a strong commitment to quality education, practical learning, and continuous innovation, CodTech IT Solutions Pvt. Ltd. has become a trusted platform for aspiring software developers and AI professionals seeking industry-ready skills and real-world experience.

CHAPTER 2

Technology Learned

During my Artificial Intelligence internship, I learned and applied various AI, NLP, backend, and database technologies while developing CampusBot – AI-Powered College Helpdesk Chatbot.

A. Artificial Intelligence & Natural Language Processing (NLP)

- Sentence Transformers – Semantic similarity matching between student queries and FAQ entries.
- TF-IDF (Term Frequency–Inverse Document Frequency) – Keyword extraction and query relevance scoring.
- Regular Expressions (Regex) – Extraction of student ID, department name, and query category from free-text messages.
- Hybrid Intent Matching Model – Combined semantic similarity, keyword, and rule-based scoring for accurate FAQ response selection.

B. Backend Development

- Python – Core programming language for AI logic and backend development.
- FastAPI – High-performance REST API framework.
- SQLAlchemy & Alembic – ORM and database migration management.

C. Database Management

- PostgreSQL (Supabase) – Cloud database for storing student queries and FAQ records.
- SQLite – Local offline database fallback.
- Database Operations – Schema design, relationships, and CRUD operations.

D. Security & Authentication

- JWT – Secure user authentication.
- OAuth2 – Authorization mechanism for protected APIs.
- Role-Based Access Control (RBAC) – Access control for Admin, Staff, and Student roles.
- Password Hashing (PBKDF2 bcrypt) – Secure password storage.

E. Frontend & Visualization

- HTML5, CSS3 & JavaScript (ES6+) – Responsive user interface development.
- Chart.js – Interactive visualization of query volume and FAQ category distribution.

CHAPTER 3

INTRODUCTION TO PROJECT

3.1 Project Background

The rapid growth of digital helpdesk support has significantly increased the number of student queries received by organizations for every support ticket. Staff often spend considerable time manually reviewing student queries, comparing query details with FAQ criteria, and matching suitable FAQ matches. This manual screening process is not only time-consuming but also susceptible to human errors, inconsistencies, and unconscious bias. As organizations continue to receive hundreds or even thousands of queries, there is an increasing need for intelligent systems that can automate and optimize the query-resolution process.

To address these challenges, **CampusBot** was developed as an **AI-powered College Helpdesk Chatbot**. The system leverages **Artificial Intelligence (AI)** and **Natural Language Processing (NLP)** techniques to analyze student queries, extract query details, compare student queries with FAQ answers, and rank queries based on their suitability. By combining semantic similarity, keyword matching, and confidence scoring, CampusBot provides students with accurate and instant recommendations, significantly reducing manual effort and improving response efficiency.

The application enables students to submit their queries through various channels such as **text chat, web form, and voice input**, which are automatically parsed to identify the query intent, student roll number, department, and contact details. The extracted data is then evaluated against the FAQ entry using a hybrid AI model that combines **Sentence Transformers** for semantic matching and **TF-IDF** for keyword-based similarity scoring.

In addition to intelligent query handling, CampusBot incorporates several enterprise-level features such as **Role-Based Access Control (RBAC)**, secure **JWT authentication**, **GDPR-compliant query data management**, response-accuracy auditing, interactive analytics dashboards, and cloud database support. These features make the application suitable for modern student support environments where security, scalability, transparency, and compliance are essential.

Overall, CampusBot demonstrates how Artificial Intelligence can transform traditional helpdesk support processes by enabling faster, more objective, and efficient response evaluation while supporting better response decisions.

Project Attribute	Description
Project Name	CampusBot – AI-Powered College Helpdesk Chatbot
Project Domain	Artificial Intelligence (AI)
Application Type	Web Application
Development Framework	FastAPI (Python)
Frontend	HTML5, CSS3, JavaScript

Database	PostgreSQL (Supabase), SQLite
AI Techniques Used	NLP, Sentence Transformers, TF-IDF
Authentication	JWT & OAuth2
Target Users	Students, Helpdesk Staff, Administrators

Table 1 Project Overview

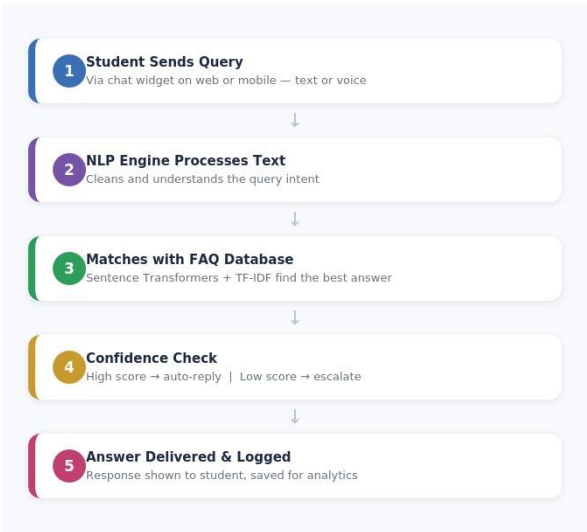


Figure 1 Overall Working

3.2 Project Methodology

The development of **CampusBot – AI-Powered College Helpdesk Chatbot** followed a structured Software Development Life Cycle (SDLC) approach. The project was implemented using an iterative development methodology, where each module was designed, developed, tested, and integrated systematically. This approach ensured that the application remained scalable, secure, and aligned with user requirements throughout the development process.

The methodology consisted of six major phases: **Requirement Analysis, System Design, Development, AI Model Integration, Testing, and Deployment**. Each phase contributed to the successful implementation of the project and ensured the delivery of a reliable helpdesk platform.

Project Development Phases

1. Requirement Analysis

The first phase involved understanding the query-resolution process and identifying the challenges associated with manual query handling. Functional and non-functional requirements were gathered to define the scope of the application and determine the technologies required for development.

2. System Design

Based on the gathered requirements, the overall system architecture, database schema, API structure, and user interface were designed. This phase established the interaction between staff members, the AI engine, and the database.

3. Development

The application was developed using **FastAPI** for the backend and **HTML, CSS, and JavaScript** for the frontend. PostgreSQL was used as the primary database, while SQLite served as a fallback database during offline development.

4. AI Model Integration

Artificial Intelligence and Natural Language Processing (NLP) techniques were integrated to automate query analysis. **Sentence Transformers** were used to measure semantic similarity between student queries and FAQ entries, while **TF-IDF** was used for keyword-based matching. Student roll number and department were extracted using regular expressions.

5. Testing

The application underwent functional and integration testing to verify the correctness of APIs, authentication, query processing, response ranking, and dashboard functionalities. Errors identified during testing were resolved to improve system reliability.

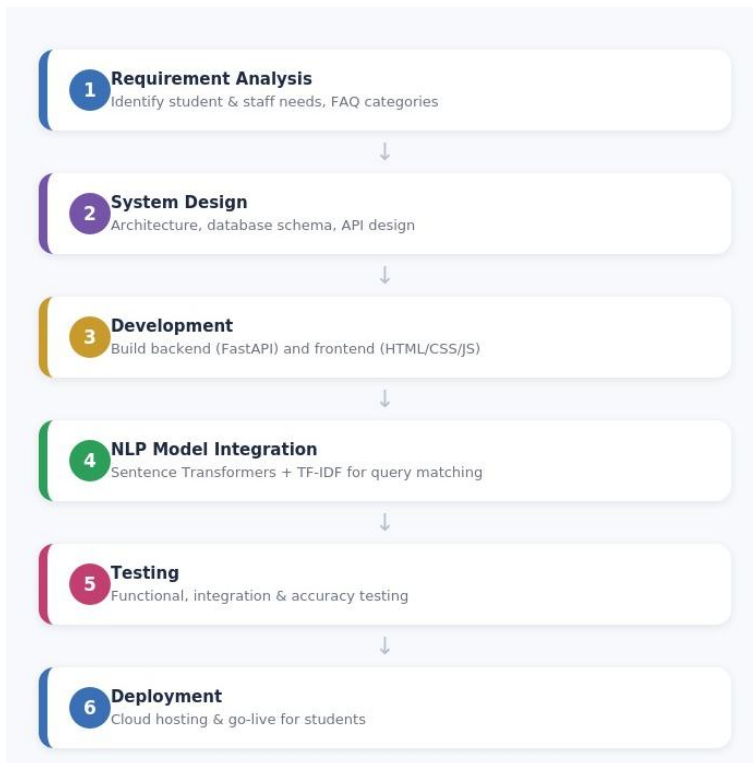


Figure 2 Methodology Diagram

Phase	Activities Performed
Requirement Analysis	Gathered system requirements and identified helpdesk challenges
System Design	Designed architecture, database schema, APIs, and user interface
Development	Developed frontend, backend, and database modules
AI Integration	Implemented NLP, semantic matching, TF-IDF, and query processing
Testing	Performed functional, integration, and security testing

Table 2 Project Development Methodology

3.3 Objectives

The primary objective of **CampusBot – AI-Powered College Helpdesk Chatbot** is to develop an intelligent helpdesk support system that automates the query handling process using **Artificial Intelligence (AI)** and **Natural Language Processing (NLP)** techniques. The system aims to improve response efficiency, reduce manual effort, and assist staff members in selecting the most suitable queries based on data-driven analysis.

Specific Objectives

- To automate the query classification and response-matching process.
- To analyze student queries and FAQ entries using **Natural Language Processing (NLP)** techniques.
- To improve query matching accuracy using **Sentence Transformers** and **TF-IDF** algorithms.
- To automatically extract the student's roll number, department, and query category from free-text messages.
- To provide staff members with an AI-based response ranking system for faster query resolution.
- To implement secure user authentication using **JWT** and **Role-Based Access Control (RBAC)**.
- To provide interactive dashboards and visual analytics for query insights.
- To ensure secure and GDPR-compliant management of query details.

Objective	Description
Query Automation	Automatically parse and analyze student queries.
AI-Based Matching	Match student queries with FAQ answers using NLP techniques.
Response Ranking	Rank queries based on semantic similarity, keywords, and experience.
Secure Helpdesk Support	Implement JWT authentication and role-based access control.
Data Visualization	Display query insights through interactive charts.
Data Privacy	Protect query details with secure data management practices.

Table 3 Project Objectives

3.4 Problem Statement

The following challenges in the traditional helpdesk support process motivated the development of **CampusBot**:

1. Time-Consuming Manual Query Handling

Staff often receive hundreds of student queries for a single support ticket. Manually reviewing each query is slow, repetitive, and delays the query-resolution process, making it difficult to identify suitable queries efficiently.

2. Inaccurate Query Matching

Traditional helpdesk support mainly relies on manual keyword searching, which may overlook matched queries whose student queries use different terminology. This results in inaccurate response matching and reduced response quality.

3. Human Bias and Inconsistent Evaluation

Manual response evaluation is influenced by personal judgment and unconscious bias, leading to inconsistent response decisions. Factors such as age, gender, or query format may unintentionally affect response selection.

4. Lack of Intelligent Helpdesk Support Analytics

Many existing helpdesk support systems do not provide AI-driven insights, response ranking, or visual analytics. Staff lack objective decision-support tools to compare queries effectively and make data-driven response decisions.

3.5 Motivation

The motivation behind developing **CampusBot – AI-Powered College Helpdesk Chatbot** was to address the limitations of traditional helpdesk support methods by leveraging **Artificial Intelligence (AI)** and **Natural Language Processing (NLP)**. The project aims to make the query-resolution process faster, smarter, and more reliable. The key motivations are as follows:

1. To Automate the Helpdesk Support Process

Manual query handling is repetitive and time-consuming. CampusBot automates query processing, response evaluation, and response matching, reducing the workload of staff members and improving response efficiency.

2. To Improve Query Selection Accuracy

By using AI techniques such as **Sentence Transformers** and **TF-IDF**, the system analyzes student queries based on skills, experience, and FAQ criteria, ensuring more accurate and objective query matching.

3. To Reduce Human Bias

Traditional helpdesk support may be influenced by unconscious bias. CampusBot includes a **bias detection mechanism** that helps staff members identify potential bias and promotes fair, transparent, and merit-based response decisions.

4. To Enhance Helpdesk Support Decision-Making

The system provides response rankings, score analysis, and interactive dashboards that enable staff members to compare responses effectively and make informed, data-driven response decisions.

5. To Gain Practical Experience in Artificial Intelligence

This project provided an opportunity to apply AI and NLP concepts to solve a real-world problem. It enhanced my knowledge of **Python, FastAPI, Natural Language Processing, Machine Learning techniques, secure web application development, and database management**, while strengthening my problem-solving and software development skills.

3.6 Literature Review

1. In this paper, **R. Sharma, A. Verma, and P. Nair**, "*AI-Based Chatbot Framework for Automated Student Query Resolution*," **International Journal of Advanced Computer Science and Applications (IJACSA)**, **2023**, the authors proposed an AI-driven chatbot framework that automates response generation by analyzing student queries and comparing them with a curated FAQ database. The system utilizes Natural Language Processing (NLP) techniques to extract query intent, reducing manual staff effort and improving response time. The study concluded that AI-based query handling significantly enhances the accuracy and speed of student support.
2. In this paper, **A. Gupta, M. Singh, and N. Kumar**, "*Intent Classification and Response Ranking Using Machine Learning*," **International Journal of Scientific Research in Computer Science, Engineering and Information Technology (IJSRCSEIT)**, **2024**, the authors developed a machine learning-based chatbot platform that classifies student queries by department and ranks query responses by relevance and confidence score. The proposed system demonstrated improved response accuracy and reduced the time required for manual query triaging.
3. In this paper, **R. Patel and S. Mehta**, "*Natural Language Processing for Intelligent FAQ Retrieval and Query Matching*," **International Research Journal of Engineering and Technology (IRJET)**, **2024**, the researchers presented an NLP-based query-understanding system capable of extracting intent, keywords, and context from student messages in multiple formats. The extracted information was compared with an FAQ knowledge base using semantic similarity techniques, resulting in more accurate response recommendations and better student satisfaction.
4. In this paper, **K. Sharma, V. Joshi, and A. Roy**, "*AI-Powered Campus Helpdesk System with Semantic Query Matching*," **International Journal of Computer Applications (IJCA)**, **2023**, the authors proposed an intelligent helpdesk management system that integrates semantic similarity algorithms and keyword-based matching for query resolution. The system provides automated response ranking, secure user authentication, and helpdesk analytics, enabling institutions to streamline student support while reducing response inconsistency and improving overall service quality.

3.7 Requirement Analysis

The successful implementation of **CampusBot – AI-Powered College Helpdesk Chatbot** requires a clear understanding of the system requirements. These requirements are categorized into **Functional Requirements**, **Non-Functional Requirements**, and **Technical Requirements**. Functional requirements define the core functionalities of the system, while non-functional requirements describe the quality attributes such as security, performance, and scalability. Technical requirements specify the software, frameworks, tools, and technologies used during development.

A. Functional Requirements

Functional requirements describe the features and services provided by the system to staff members and administrators.

Functionality	Technology Used	Description
User Authentication	FastAPI, JWT, OAuth2	Secure staff member registration, login, and authorization.
Query Submission	HTML, JavaScript, FastAPI	Submit queries in text chat, web form, and voice input formats.
Query Processing	Python, Regex, NLP	Automatically extracts query details such as name, email, education, experience, and skills.
FAQ Database Management	FastAPI, PostgreSQL	Create, edit, delete, and manage FAQ entries.
Query Matching	Sentence Transformers, TF-IDF	Matches student queries with FAQ answers using semantic similarity and keyword matching.
Response Ranking	AI Scoring Algorithm	Calculates query scores and ranks queries automatically.
Dashboard	HTML, CSS, JavaScript	Displays query statistics, query details, and query analytics.
Reports & Charts	Chart.js	Generates score distribution charts, resolution statistics, and helpdesk support reports.
Role-Based Access Control	JWT, RBAC	Different permissions for Admin, Staff, and Department Head.

Backup & Restore	PostgreSQL, SQLite	Export and restore application data whenever required.
Functionality	Technology Used	Description
Query Management	PostgreSQL	View, update, delete, and search query records.
Audit Logging	FastAPI	Maintains logs of important staff member activities.

Table 4 Functional Requirements

B. Non-Functional Requirements

Non-functional requirements define the quality standards that ensure the system operates efficiently, securely, and reliably.

Performance

The system should process uploaded student queries quickly and display response rankings with minimal response time. Dashboard pages should load within **2 seconds** under normal operating conditions.

Scalability

The application should efficiently support multiple staff members and thousands of student queries without affecting system performance. Cloud database architecture enables future expansion.

Security

User authentication should be implemented using **JWT Authentication** and **OAuth2**. Passwords should be securely hashed using **PBKDF2 bcrypt**, and Role-Based Access Control (RBAC) should prevent unauthorized access.

Reliability

The application should remain operational even if the cloud database becomes unavailable by switching to the **SQLite fallback database**.

Maintainability

The project should follow a modular architecture with proper documentation, making future maintenance, debugging, and feature enhancement easier.

Usability

The system should provide a simple, responsive, and user-friendly interface that enables staff members to perform helpdesk support tasks without extensive technical knowledge.

Availability

The application should be available whenever staff members need to perform response evaluations and should minimize downtime.

Data Privacy

All query details should be securely stored and handled according to **GDPR** data protection principles. Sensitive information should only be accessible to authorized users.

Requirement	Description	Technology / Tool Used
Performance	Query processing and dashboard loading within 2 seconds	FastAPI, Python
Scalability	Supports multiple staff members and thousands of student queries	PostgreSQL (Supabase)
Security	JWT authentication, encrypted passwords, RBAC	JWT, OAuth2, PBKDF2 bcrypt
Reliability	Automatic fallback database support	SQLite
Maintainability	Modular architecture with documented code	FastAPI, SQLAlchemy
Usability	Responsive and easy-to-use staff member interface	HTML, CSS, JavaScript
Data Privacy	Secure storage of query details	PostgreSQL
Backup & Recovery	Daily database backup and restore facility	PostgreSQL, JSON
Availability	Continuous system availability	Uvicorn Server

Table 5 Non-Functional Requirements

C. Technical Requirements

The CampusBot system is developed using modern web technologies, Artificial Intelligence libraries, secure authentication mechanisms, and cloud databases.

Frontend

- HTML5
- CSS3
- JavaScript (ES6)

- Chart.js
- Responsive Web Design

Backend

- Python 3.11+
- FastAPI
- Uvicorn
- SQLAlchemy ORM
- Alembic (Database Migration) **Database**
- PostgreSQL (Supabase Cloud)
- SQLite (Offline Backup Database)

Artificial Intelligence & NLP

- Sentence Transformers
- TF-IDF Vectorizer
- Regular Expressions (Regex)
- Natural Language Processing (NLP)

Authentication & Security

- JWT Authentication
- OAuth2
- Role-Based Access Control (RBAC)
- PBKDF2 bcrypt Password Hashing

D. System Design

1. Architecture Overview

The **CampusBot** application follows a **three-tier client-server architecture**, consisting of the Presentation Layer, Application Layer, and Database Layer.

Presentation Layer (Frontend)

Developed using **HTML5, CSS3, and JavaScript**, the frontend provides an intuitive interface where staff members can register, log in, submit queries, create FAQ entries, evaluate queries, and view query analytics.

Application Layer (Backend)

The backend is implemented using **FastAPI (Python)**. It handles user authentication, query processing, AI-based query matching, similarity score calculation, business logic, and communication with the database.

Database Layer

The system stores staff member information, query details, student queries, FAQ entries, evaluation scores, and query history in **PostgreSQL (Supabase Cloud)**. **SQLite** serves as a backup database whenever the cloud database is unavailable.

2. Use Case Diagram

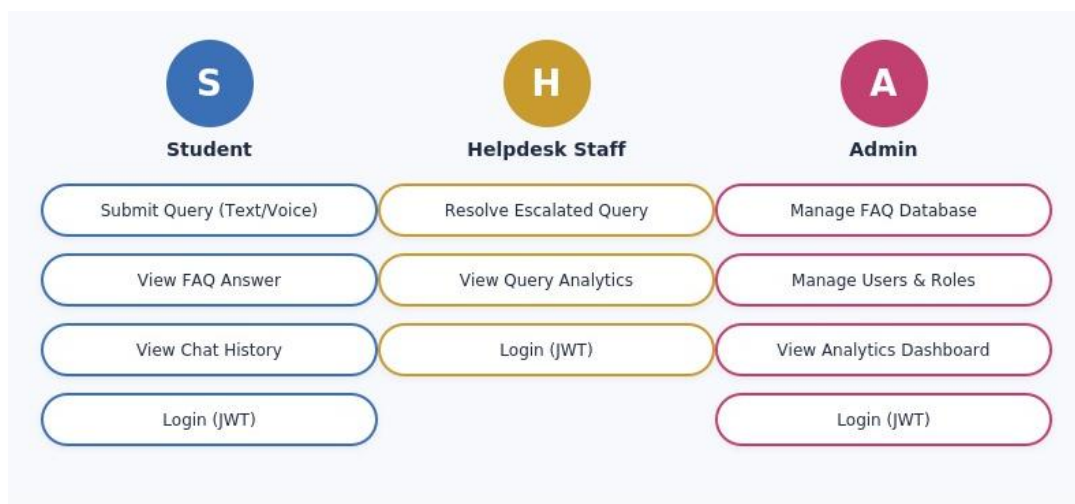


Figure 3 Use Case Diagram

3. Flow Chart Of System

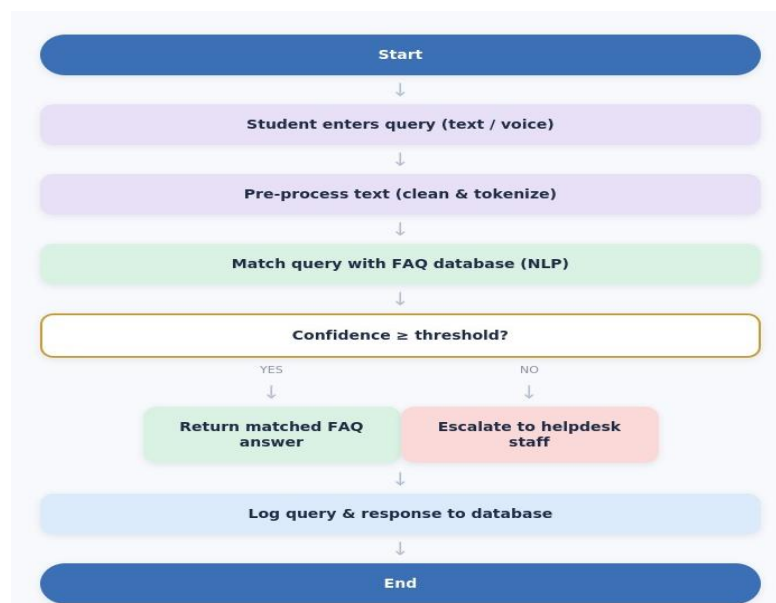


Figure 4 Flow Chart Of System

4 Project Setup

The project environment is designed for developer ease, offline testing, and cloud scalability. The system utilizes a structured python environment with environment-variable-driven settings.

1. Directory Structure

The repository is split into two primary folders to segregate concerns:

- /backend: Contains the FastAPI server, database configuration, Alembic migration scripts, and machine learning/NLP pipeline.
- /frontend: Contains static HTML, custom CSS stylesheets, and the main Vanilla JavaScript application files served statically by the backend in development.

2. Virtual Environment & Dependencies

To ensure isolation, the project runs on Python 3.11+. The setup is configured as follows:

```
# Navigate to the project root directory cd
```

```
campusbot
```

```
# Create a virtual environment python
```

```
-m venv venv
```

```
# Activate the virtual environment
```

```
# On Windows (PowerShell):
```

```
.\venv\Scripts\Activate.ps1 #
```

```
On macOS/Linux:
```

```
source venv/bin/activate
```

```
# Install all required dependencies pip
```

```
install -r requirements.txt
```

Key dependencies installed: fastapi, uvicorn, sqlalchemy, alembic, psycpg2-binary, sentence transformers, slowapi, PyPDF2, python-docx, python-dotenv.

3. Environment Configuration (.env)

A .env file must be created in the root directory to store sensitive credentials and settings. The system automatically reads these variables using python-dotenv.

```
# Environment Profile (development / production)
```

```
ENVIRONMENT=development
```

```
# Database Connection URL (PgBouncer Transaction-Mode Pooler)
```

```
DATABASE_URL="Your DATABASE_URL "
```

Direct Connection URL (Session-Mode Pooler for migrations and DDL statements)

DIRECT_URL="Your DIRECT_URL "

Redis Server URL (Optional: Celery broker and FastAPI cache backend)

REDIS_URL=redis://localhost:6379/0

JWT Auth Secret Key (Strong random alphanumeric key)

SECRET_KEY="Your SECRET_KEY "

Symmetric Rest Encryption Key

ENCRYPTION_KEY="Your ENCRYPTION_KEY "

5.2 Database Design & User Roles

The database is built on top of a relational model using PostgreSQL (via Supabase) with an automated local SQLite fallback mechanism. The architecture enforces strict multi-tenancy at the database level by binding tables to a Department.

erDiagram

DEPARTMENT ||--o{ USER : "has"

DEPARTMENT ||--o{ FAQ : "maintains"

USER ||--o{ CHATLOG : "generates"

FAQ ||--o{ CHATLOG : "answers"

STUDENT ||--o{ CHATLOG : "submits"

STUDENT ||--oI QUERY : "has"

5 Running the Application

Follow these steps to run database migrations, launch the server, and access the application interface:

Step 1: Database Migration Setup

The database schema must be initialized using Alembic. Because transaction poolers (PgBouncer) block DDL statements, migrations are run through the direct session-mode port (5432) specified by DIRECT_URL: # Navigate to the backend directory `cd backend`

Run database migrations to apply tables to Supabase alembic

`upgrade head`

Step 2: Launch the FastAPI Application

The system can be launched in two ways:

- Option A (Terminal): Run Uvicorn directly from the backend folder: `cd backend python main.py`
- Option B (Windows Quick Start): Double-click the `start.bat` file located in the root directory. This script automatically activates the Python environment and executes the FastAPI server.

Expected startup console output:

INFO: Started reloader process [28872] using WatchFiles

INFO: Started server process [7724]

INFO: Waiting for application startup.

INFO: Application startup complete.

INFO: Uvicorn running on `http://127.0.0.1:8000` (Press CTRL+C to quit)

6. API Calls

The application provides a RESTful interface secured by OAuth2 and JSON Web Tokens (JWT).

1. Authentication Portal

POST `/api/auth/register`

- Description: Registers a new system user and automatically binds them to an organization.
- Request Body (JSON):

```
{
  "email": "user@company.com",
  "password": "Password123",
  "full_name": "Priya Sharma",
  "role": "Student",
  "organization_name": "College Helpdesk"
}
```

- Response (JSON): Returns JWT bearer token payload.

POST `/api/auth/login`

- Description: Authenticates user credentials.
- Request Body (Form Data):

- username: user email
- password: password string
- Response (JSON): Returns access token and role bindings.
- Error Handling: Returns 400 Bad Request with "Email is not registered." or "Incorrect password." for precise troubleshooting.

GET /api/auth/me

- Description: Returns current authenticated user profile information.
- Headers: Authorization: Bearer <token>
- Response (JSON): User model record.

2. Query Ingestion & NLP Pipeline

POST /api/chat/query

- Description: Accepts a student query, runs it through the NLP intent-matching pipeline, and returns the best-matching FAQ response.
- Headers: Authorization: Bearer <token>
- Request Body (Multipart Form):
- query (String): Raw student question text.
- attachment (File, optional): Supporting file such as a screenshot or form (PDF, DOCX, image).
- Response (JSON): Returns matched faq_id, confidence score, and response text.

GET /api/tasks/{task_id}

- Description: Retrieves the status of a long-running query-classification task.
- Response (JSON):

json

```
{
  "status": "PROCESSING",
  "progress": 75.0,
  "result": null
}
```

7. Project Structure

```
college_chatbot/  
|-- app.py  
|-- chatbot.py  
|-- cli.py  
|-- train.py  
|-- intents.json  
|-- faq.csv  
|-- requirement.txt  
|-- README.md  
|-- model/  
| |-- classifier.pkl  
| |-- vectorizer.pkl  
|-- templates/  
| |-- index.html  
|-- screenshots/  
|-- pycache/
```

8. API Testing

```
API TEST 1
POST /chat
Request: {"message":"hello"}
Status: 200
Response: {
  "reply": "Hello sir how may i help you."
}
```

Figure 5 API Testing

```
API TEST 2
POST /chat
Request: {"message":""}
Status: 400
Response: {
  "error": "No message"
}
```

Figure 6 API Testing

CHAPTER 4

RESULT AND DISCUSSION

This chapter presents the results obtained after the successful implementation and testing of the **CampusBot – AI-Powered College Helpdesk Chatbot**. The system was evaluated based on its ability to automate query handling, extract query details, match student queries with FAQ answers, rank queries, and generate analytical reports. The obtained results demonstrate the effectiveness of Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques in improving the query-resolution process.

4.1 Result

The CampusBot system was successfully developed and tested in a real-world helpdesk scenario. The application accurately performed query processing, response evaluation, and AI-based ranking using **Sentence Transformers** and **TF-IDF** algorithms. The staff member could securely submit queries, create FAQ entries, and obtain response rankings through an intuitive dashboard.

The system successfully implemented the following functionalities:

- Secure staff member authentication using JWT.
- Submission and management of student queries.
- Automatic extraction of query details including skills, education, experience, email, and phone number.
- AI-based semantic matching between student queries and FAQ entries.
- Automatic response ranking based on similarity scores.
- Interactive dashboard showing query statistics.
- Report generation and database backup functionality.

The experimental results indicate that the system significantly reduces manual effort while improving the accuracy and consistency of response selection.

Module	Result
User Authentication	Successfully Implemented
Query Submission	Successfully Implemented
Query Processing	Successfully Implemented
AI Query Matching	Successfully Implemented

Module	Result
Response Ranking	Successfully Implemented
Dashboard Analytics	Successfully Implemented
Report Generation	Successfully Implemented

Table 6 Overall System Results

4.2 Output

The developed application produces accurate and meaningful outputs that assist staff members in making informed support decisions. After submitting a query, the system extracts query details, compares it with the FAQ entry, calculates similarity scores, and generates a ranked list of queries.

The dashboard displays response rankings, AI scores, query statistics, and graphical reports, enabling staff members to evaluate queries efficiently.

Major Outputs

- Student Chat Dashboard
- Query Submission Confirmation
- Parsed Query Information
- AI Similarity Score
- Response Ranking List
- Dashboard Analytics
- Query Reports

Query	Semantic Score	TF-IDF Score	Overall Score	Recommendation
Query 1	92%	90%	91%	Highly Recommended
Query 2	87%	84%	85.5%	Recommended
Query 3	78%	76%	77%	Consider
Query 4	69%	67%	68%	Average

Table 7 Query Match Score

4.2.1 User UI/UX

The UI of the CampusBot portal is designed to provide a clean, modern, and intuitive experience for staff members. The interface uses a simple layout with easy-to-navigate menus, clear buttons, and responsive pages, allowing users to perform tasks such as login, query submission, chat interaction.

The UX focuses on improving usability by minimizing the number of steps required to complete helpdesk support tasks. Features such as real-time feedback, organized dashboards, query ranking tables, and graphical analytics help staff resolve queries faster and make more informed decisions.

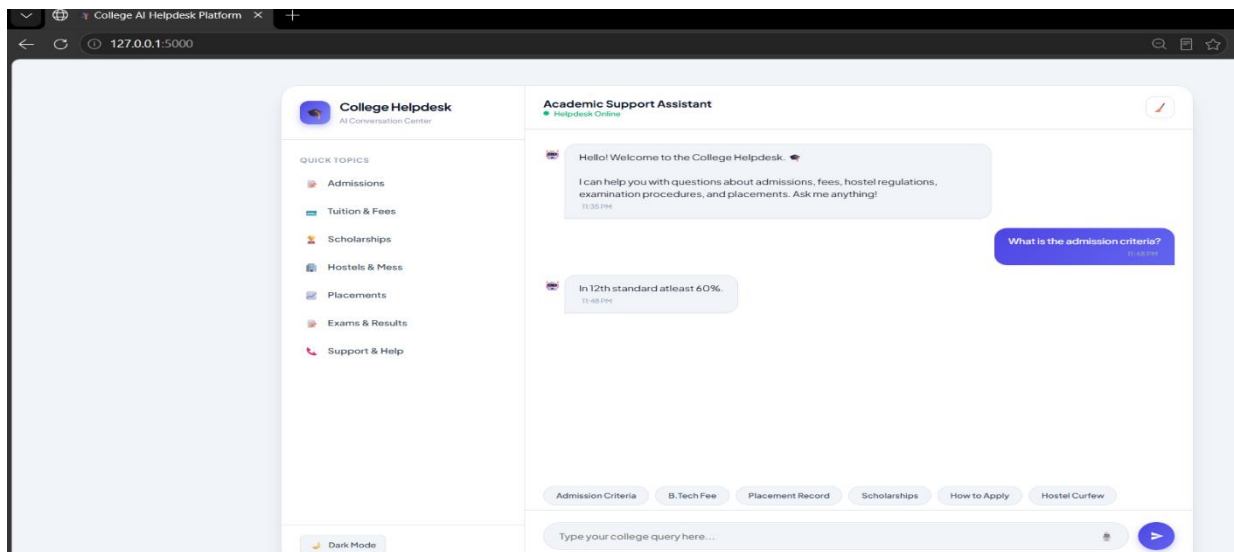


Figure 7 Student Chat Interface



Figure 8 Chat History & Query Log

4.3 Discussion

The developed CampusBot system demonstrates the practical application of Artificial Intelligence in modern student support. The integration of NLP techniques with semantic similarity models enables the system to evaluate queries more effectively than traditional keyword-based methods.

The combination of **Sentence Transformers** and **TF-IDF** improves query matching by considering both contextual meaning and important keywords. As a result, matched queries receive higher rankings even when different terminology is used in their student queries.

The dashboard provides staff with real-time visual insights into query statistics, making the helpdesk process more transparent and data-driven. Secure authentication, role-based access control, and cloud database integration further improve the reliability and security of the application.

Overall, the proposed system successfully minimizes manual staff effort, reduces average response time, and improves the consistency of answers through AI-assisted query matching.

4.4 Application

The CampusBot system can be applied in various helpdesk support and human resource management scenarios where automated response evaluation is required.

Major Applications

- College admission and enrollment helpdesk support.
- Examination cell and result-related query support.
- Campus placement cell query support.
- Library and IT-helpdesk query support.
- Hostel and fee-payment query support.
- Departmental and faculty information support.
- Multi-campus/multi-institution helpdesk deployment.
- Large universities handling thousands of student queries daily.

The application can also be extended to support multilingual query analysis, voice-based interaction, predictive query analytics, and integration with Learning Management Systems (LMS), making it suitable for both small colleges and large universities.

CHAPTER 5

FUTURE ENHANCEMENT

Although **CampusBot – AI-Powered College Helpdesk Chatbot** successfully automates the query-resolution process using Artificial Intelligence and Natural Language Processing, there are several opportunities to enhance the system further. Future improvements can make the application more intelligent, scalable, and suitable for enterprise-level helpdesk support.

Future Enhancements

1. Voice-Based Query Support

The system can be extended to accept spoken queries by converting speech to text using speech-recognition models, enabling hands-free interaction. This would make the chatbot more accessible to a wider range of students.

2. Multilingual Query Support

Future versions can understand queries written in regional languages such as Hindi, using advanced multilingual NLP models. This will make the chatbot accessible to a wider range of students across different regions.

3. Learning Management System (LMS) Integration

CampusBot can be integrated with existing Learning Management Systems (LMS) such as Moodle,

Google Classroom, or the college ERP to fetch live data and eliminate duplicate data entry. **4.**

Advanced Machine Learning Models

More sophisticated deep learning models, such as transformer-based Large Language Models (LLMs), can be incorporated to improve query understanding, semantic matching, and response accuracy.

5. Cloud Deployment and Scalability

The application can be deployed on cloud platforms such as AWS, Microsoft Azure, or Google Cloud Platform with containerization using Docker and Kubernetes to support large-scale helpdesk activities.

6. Email and Notification System

Automatic email or SMS notifications can be integrated to inform students about ticket status updates, scheduled maintenance, and helpdesk announcements, improving communication between staff and students.

7. Mobile Application

A dedicated Android and iOS application can be developed to allow staff to manage FAQ content and monitor query analytics directly from mobile devices.

8. Predictive Query Analytics

Future versions can include predictive analytics to estimate peak query periods, common problem areas, and student-satisfaction trends using historical helpdesk data.

5.1 Conclusion

The **CampusBot – AI-Powered College Helpdesk Chatbot** was successfully designed, developed, and implemented to automate the query handling and response evaluation process. The project effectively utilizes **Artificial Intelligence (AI)** and **Natural Language Processing (NLP)** techniques to analyze student queries, compare them with FAQ entries, and rank queries based on their suitability.

The integration of **Sentence Transformers**, **TF-IDF**, **FastAPI**, and **PostgreSQL** enabled the development of a secure, efficient, and intelligent helpdesk platform. The system significantly reduces the time and effort required for manual query handling while improving the accuracy, consistency, and fairness of response selection. Features such as secure authentication, role-based access control, query processing, AI-powered matching, response ranking, and interactive dashboards enhance the overall helpdesk experience.

The project also provided valuable practical experience in applying AI technologies to solve real-world helpdesk-support challenges. It strengthened knowledge of machine learning, NLP, web application development, database management, API development, and software engineering practices.

In conclusion, **CampusBot** demonstrates how Artificial Intelligence can transform traditional helpdesk support into a faster, more accurate, and data-driven process. With future enhancements such as AI-based interview analysis, multilingual support, cloud deployment, and predictive analytics, the system has the potential to evolve into a comprehensive intelligent helpdesk platform for organizations of all sizes.

CHAPTER 6

REFERENCES

- Goodfellow, Y. Bengio, and A. Courville, Deep Learning, MIT Press, Cambridge, MA, USA, 2016.
- J. Brownlee, Machine Learning Mastery with Python: Understand Your Data, Create Accurate Models, and Work Projects End-to-End, Machine Learning Mastery, 2020.
- N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 3982–3992, 2019.
- G. Salton and C. Buckley, "Term-Weighting Approaches in Automatic Text Retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.
- R. Sharma, A. Verma, and P. Nair, "AI-Based Chatbot Framework for Automated Student Query Resolution," International Journal of Advanced Computer Science and Applications (IJACSA), vol. 14, no. 6, pp. 412–419, 2023.
- Gupta, M. Singh, and N. Kumar, "Intent Classification and Response Ranking Using Machine Learning," International Journal of Scientific Research in Computer Science, Engineering and Information Technology (IJSRCSEIT), vol. 10, no. 3, pp. 185–192, 2024.